Autonomous Vehicle Challenge Documentation

# Advice

## General

- Don't use most of the code outside of the "useful code" folder. The "other code" folder is there for reference, but it is messy and undocumented because it didn't work and lots of it was somewhat rushed as we pivoted from SLAM to a simpler solution. The stuff in the designated folder will save you a lot of time assuming you can fix problems with the lidarInterface.

- The stuff in main.py might be a useful reference for how we did some of our logging, but don't try to get main.py working as some code has been moved around in other files and main.py hasn't been updated to reflect these changes.

- If you want to try your hand at implementing SLAM, I HIGHLY recommend Dr. Cyril's lectures and videos on Youtube. If you do go for this, be aware that you not put all your eggs in one basket, and have some of your team members pursue another solution in parallel in case it doesn't work out.

### – YOU DON'T NEED A CAR TO BEGIN TESTING

- We wish we had thought to do this sooner, but you can strap your sensors to a bucket and push it around manually to simulate your car. We recommend doing this over taking a simulation route since, speaking from experience, translating what you have from a simulation to real-world comes with its own challenges.

## Object Recognition Model

- We have a fine-tuned YoloV11 to recognize ramps as well as blue, green, red, and yellow buckets. Each bucket color has its own label, and that is what we use to get the color of the bucket.

- If you continue with our solution of using a different label for each bucket color, you need to make it much more robust than we did. When we arrived at the competition, their blue buckets were a different shade of blue than the ones we trained on, so we had to scramble to train a new model on the new color. You will need to train on various hues and in various levels of shading to have a sufficiently robust model for the competition.

- Another potentially better solution, however, is to train the model on just the labels "bucket" and "ramp". By simply using a single label for all the buckets and training on a few different colors, your model will focus on the shape of a bucket,

and not specific colors. You can then use the boxes around the objects you detect to read pixels inside that box and find the color that way.

**Lidar**

- We haven't figured out why, but the lidar interface often registers the data coming out of the lidar as incorrect. When this happens, the lidar must be restarted in order to send out data which the interface considers "correct". This process takes 2-3 seconds, and often has to be done multiple times before it works. Unless you write asynchronous or multithreaded code, your program will freeze during this time while waiting for the lidar to restart, which WILL cause you problems. We are using a custom rplidar library which has been modified slightly from the original. You should probably first try to get the original library working before trying anything else.

## The Winning Team's Method

- There were 7 teams at the 2025 AVC challenge in Morrilton. Out of these 7 teams, only one of them was able to fully complete the course. Our team placed 2nd, but it wasn't because of our program working correctly. We only circled one blue bucket correctly, but we got 50 points for having the best team notebook, which gave us enough points to tie for 2nd place. We as the CS team take responsibility for this and understand that we need to do better.
- The team that placed 1st completed the course using a neural network. They used a video of the car correctly running the course (from the car's POV) as input and used the instructions sent to the car's motor and wheels as output. Specifically, they split the video into frames and used each frame as the input to the neural network. When the car actually ran the course, it took the video stream from the attached camera, split it into frames, and entered each frame into the neural network. The neural network would then output a wheel direction/motor speed for the car based on what was observed in the input frame.
- There are benefits and downsides to this approach. It seems relatively easy to implement and should yield good results. However, it requires that the car be driven around the course while recording before being placed on the actual course, so that the neural network can be trained. The course opens before the actual competition starts, so anyone using this method has the opportunity to drive the car around a few times while recording a video and saving the controller inputs, and then to train a neural network using this video and the inputs.
- This method is robust and should be able to handle the course obstacles being moved around a bit in between heats, just like they are supposed to be as stated in the rules of the competition.

- Some of us felt like this method was “cheating” and was not in line with the spirit of the competition, while some of us felt like it was fine. Those of us who didn’t like it believed that the vehicle wasn’t truly autonomous if you had to drive it around the course and record its inputs and outputs a few times for it to work and that it wasn’t much different from just hard-coding in the actions that the car should take. Those of us who did like it felt like it was a smart approach and cited that the vehicle is tolerant to the course obstacles being moved around, so it is “autonomous” enough. Overall, we aren’t recommending that you use this method, or recommending that you don’t use it. We just felt like you guys should be aware of the method that the winning team used last year. We think it would be more fun for you guys to come up with your own method though, because programming the car and getting it to work is fun!
- Good luck!